

EEST N°1 De Vicente Lopez
Laboratorio de Programación (LPR) — 5° 3° A-B

Informe de Proyecto

Calculadora de Edad — Actividad 1: Revisión Integral (Python & C++)

Alumno/a: Lucas Del Pino, Ulises Ferrari, Maximo Orue y Federico Mattia

Curso: 5° 3° A-B

Docente: Prof. York, Prof. Ganduglia

Fecha de entrega: 31 de agosto de 2026

Repositorio GitHub: <https://github.com/Pin0l33/CALCULADORAEDAD.git>

1. Introducción

El presente informe documenta el desarrollo de la Actividad 1 de revisión integral de la materia Laboratorio de Programación (LPR), correspondiente al 1º cuatrimestre de 2026. El objetivo del trabajo consistió en implementar, en los lenguajes Python y C++, un programa capaz de calcular la edad exacta de una persona a partir de su fecha de nacimiento, incorporando un filtro de consistencia que valide que la fecha ingresada exista realmente en el calendario (control de años bisiestos y de la cantidad de días de cada mes).

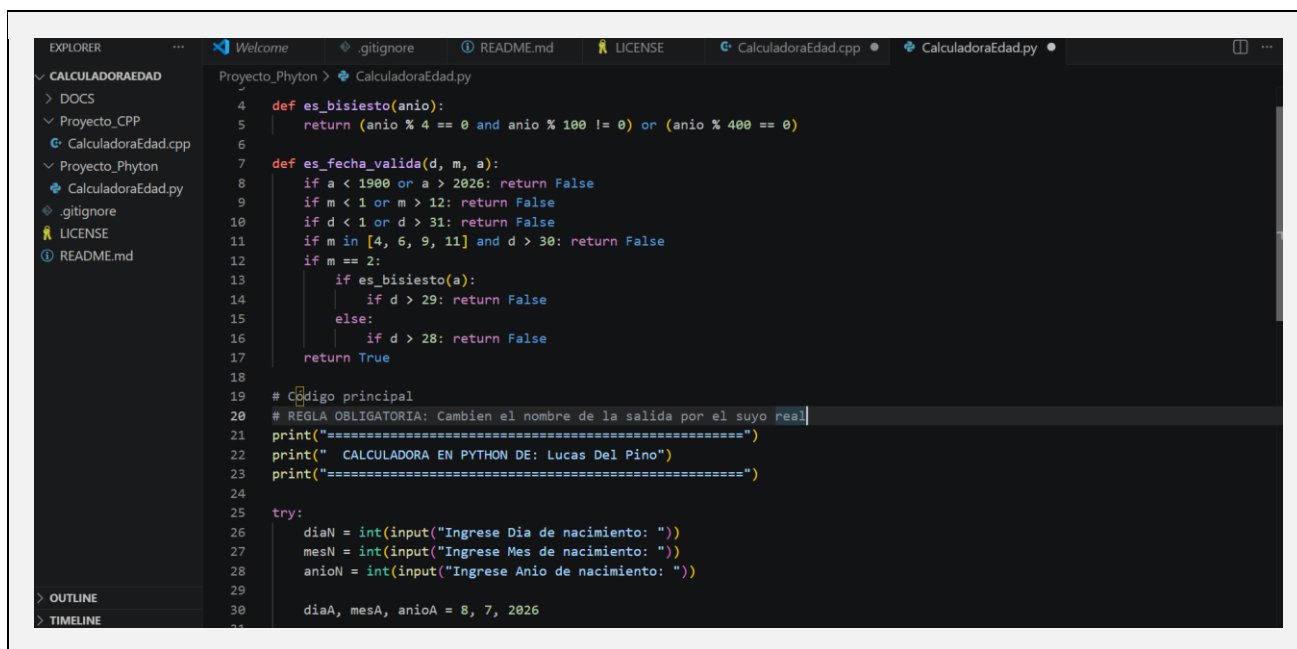
2. Estructura del repositorio

El proyecto fue organizado en GitHub siguiendo la estructura de carpetas solicitada por la cátedra:

```
/CALCULADORAEDAD/ → .gitignore, README.md, LICENSE, Docs/, Proyecto_CPP/, Proyecto_Python/
```

3. Captura del código fuente (Visual Studio Code)

A continuación se muestra la captura de pantalla del código fuente abierto en Visual Studio Code, donde se puede verificar el nombre real del alumno en la salida del programa.



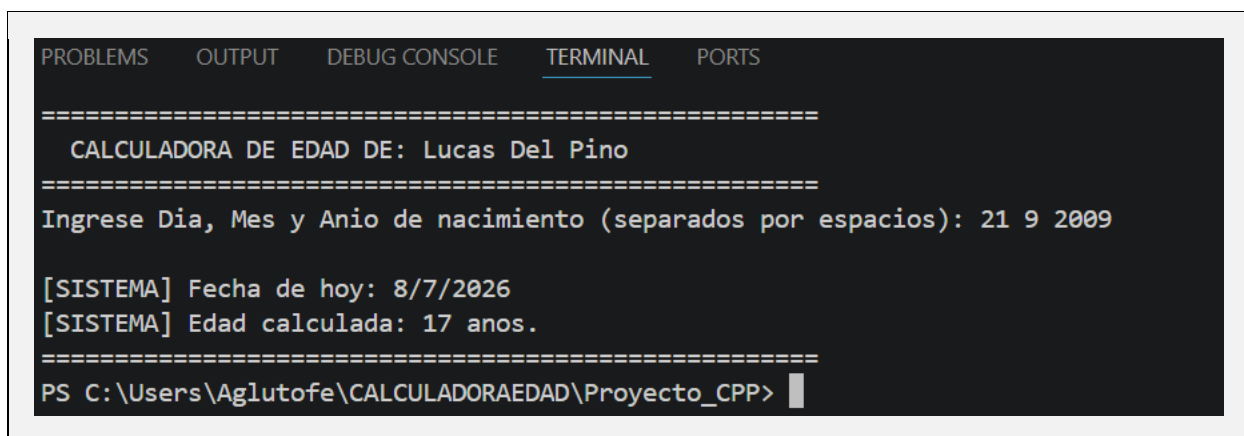
```
EXPLORER  Welcome  .gitignore  README.md  LICENSE  CalculadoraEdad.cpp  CalculadoraEdad.py

CALCULADORAEDAD
├── DOCS
├── Proyecto_CPP
├── CalculadoraEdad.cpp
├── Proyecto_Python
├── CalculadoraEdad.py
├── .gitignore
├── LICENSE
└── README.md

Proyecto_Python > CalculadoraEdad.py
4  def es_bisiesto(anio):
5      return (anio % 4 == 0 and anio % 100 != 0) or (anio % 400 == 0)
6
7  def es_fecha_valida(d, m, a):
8      if a < 1900 or a > 2026: return False
9      if m < 1 or m > 12: return False
10     if d < 1 or d > 31: return False
11     if m in [4, 6, 9, 11] and d > 30: return False
12     if m == 2:
13         if es_bisiesto(a):
14             if d > 29: return False
15         else:
16             if d > 28: return False
17     return True
18
19 # Código principal
20 # REGLA OBLIGATORIA: Cambien el nombre de la salida por el suyo real
21 print("=====")
22 print(" CALCULADORA EN PYTHON DE: Lucas Del Pino")
23 print("=====")
24
25 try:
26     diaN = int(input("Ingrese Dia de nacimiento: "))
27     mesN = int(input("Ingrese Mes de nacimiento: "))
28     anioN = int(input("Ingrese Anio de nacimiento: "))
29
30     diaA, mesA, anioA = 8, 7, 2026
```

4. Captura de la ejecución (Terminal)

4.1 Ejecución del programa en C++



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

=====
CALCULADORA DE EDAD DE: Lucas Del Pino
=====
Ingrese Dia, Mes y Anio de nacimiento (separados por espacios): 21 9 2009

[SISTEMA] Fecha de hoy: 8/7/2026
[SISTEMA] Edad calculada: 17 anos.
=====
PS C:\Users\Aglutofe\CALCULADORAEDAD\Proyecto_CPP>
```

4.2 Ejecución del programa en Python

```
PS C:\Users\Aglutofe\CALCULADORAEDAD> python -u "c:\Users\Aglutofe\CALCULADORAEDAD\Proyecto_Phyton\CalculadoraEdad.py"
=====
  CALCULADORA EN PYTHON DE: Lucas Del Pino
=====
Ingrese Dia de nacimiento: 21
Ingrese Mes de nacimiento: 9
Ingrese Año de nacimiento: 2009

[SISTEMA] Fecha de hoy: 8/7/2026
[SISTEMA] Edad calculada: 17 años.
PS C:\Users\Aglutofe\CALCULADORAEDAD> █
```

5. Sección personal: variables dinámicas vs. estáticas

En Python, las variables tienen tipado dinámico: no se declara explícitamente su tipo, sino que este se infiere en tiempo de ejecución y puede cambiar si se le asigna un valor de otro tipo a la misma variable. En cambio, en C++ las variables tienen tipado estático: es obligatorio declarar el tipo de dato (int, float, bool, etc.) antes de usarlas, y ese tipo queda fijo durante toda la ejecución del programa, lo cual se verifica en tiempo de compilación. Esta diferencia hace que Python sea más flexible y rápido de escribir, mientras que C++ ofrece mayor control y detecta errores de tipo antes de ejecutar el programa.

6. Conclusión

El desarrollo de esta actividad permitió repasar estructuras de control, funciones y validación de datos, comparando su implementación en un lenguaje interpretado de tipado dinámico (Python) y en un lenguaje compilado de tipado estático (C++), consolidando los conceptos fundamentales de lógica y algoritmos trabajados durante el curso.